

Kontrollerle Etkileşim ve Olay (Event) Yönetimi

Görsel Programlama - Hafta 3

Windows Form uygulaması (.net framework)



olay ismi

{

olay gerçekleşeceğinde çalışacak kodlar

}

Programı Canlandırmak

Haftanın Başlığı: Programı Canlandırmak: Olaylar ve Etkileşim.
Bu konu, Olay G d ml  Programlama ve Etkileşimli Kontrollerin temelini oluřturur.



Pedagojik Hedefler I

Öğrenciler, 'olay' (event) kavramının ne olduğunu detaylıca anlayacaktır. Bu kavram, modern kullanıcı arayüzü programlamanın temel paradigmasıdır.



Pedagojik Hedefler II

Öğrenciler, bir programın statik bir tasarımdan nasıl dinamik ve etkileşimli bir uygulamaya dönüştüğünü kavrayacaklardır.



Pedagojik Hedefler III

En temel olay olan Button_Click olayını kullanarak pratik yapabilmek hedeflenmektedir.

TextBox'tan veri okuma ve Label'a veri yazma işlemlerini gerçekleştirebileceklerdir.



Olay (Event) Kavramı

Olay kavramı, gerçek hayattan bir analogi ile açıklanmalıdır.
Örnek: 'Kapı zilinin çalması bir olaydır'.



Olay Yönetimi

Gerçek hayattaki bu olaya tepkiniz (eyleminiz), kapıyı açmaktır.
Programlamada da olaylar, programcının belirlediği eylemleri tetikler.



Programlamada Olaylar

Programlamada olaylar, kullanıcının arayüzde gerçekleştirdiği çeşitli eylemlerdir.

Örnekler: Bir butona tıklama, fareyi bir kontrolün üzerine getirme, klavyeden bir tuşa basma.



Olay Güdümlü Programlama (OGP)

OGP, modern kullanıcı arayüzü (UI) programlamanın temel paradigmasıdır. Programın akışını olaylar belirler.



OGP Çalışma Prensipleri

Program, bir sonraki komuta geçmek yerine pasif durumda kalır ve bir kullanıcı eylemini 'bekler'.

Kullanıcı eylemi (olay) gerçekleştiğinde, program beklemeyi bırakır ve ilgili olay kod bloğunu (eylemi) çalıştırır.

Programcının Görevi

Programcının görevi, bu olaylar meydana geldiğinde hangi spesifik kodların çalışacağını önceden belirlemektir.



Bir Olayın Anatomisi

Visual Studio'da bir butona çift tıklandığında, otomatik olarak olay işleyici bir metot oluşturulur.

Metot imza incelemesi

`private void btnGiris_Click(object sender, EventArgs e)` metodu satır satır incelenmelidir.

Metot İmza Detayı I

private void ifadesi, metodun başlangıç ifadesidir.
private: Bu metoda sadece ilgili form içinden erişilebileceğini belirtir.



Metot İmza Detayı II

void anahtar kelimesi, bu metodun geriye herhangi bir değer 'döndürmediğini' belirtir.
Olay işleyici metotlar genellikle void tipindedir.

Metot İmza Detayı III

Metodun adı btnGiris_Click şeklindedir.
Standardı: [Kontrolün Adı]_[Olayın Adı] formatında otomatik olarak oluşturulur.

İsimlendirmenin Önemi

Bu isimlendirme standardı, kodun hangi kullanıcı arayüzü kontrolünün hangi olayına tepki verdiğini kolayca anlamayı sağlar.

Olay Parametreleri I

Parantez içindeki (object sender, EventArgs e) ifadeleri, olay hakkında ek bilgi taşıyan parametrelerdir.

Olay Parametreleri II

sender parametresi, olayı tetikleyen kontrolün kendisidir (örneğin, tıklanan buton nesnesi).
Tür dönüşümü yapılarak bu nesneye erişilebilir.



Olay Parametreleri III

e parametresi, olayla ilgili ek verileri içerir.
Örneğin fare koordinatları veya tuş bilgileri gibi.

Parametre Kullanımı

Başlangıç seviyesinde, bu sender ve e parametrelerinin genellikle olay kod bloğu içinde kullanılmayacağı belirtilebilir.

Süslü parantezler { ... } arasına yazılan kodlar, olay gerçekleştiğinde çalışacak olan 'eylem' listesini oluşturur.

Kullanıcı arayüzü uygulamalarının en temel işlevi veri okuma ve yazmadır.



TextBox'tan Veri Okuma I

Bir TextBox kontrolünün içindeki metni (string değeri) almak için .Text özelliği kullanılır.



TextBox'tan Veri Okuma II

Örnek kod satırı: `label1.Text = txtKullaniciAdi.Text;`



TextBox'tan Veri Okuma III

Bu kod satırı, txtKullaniciAdi isimli metin kutusunun içerisindeki yazıyı okur.

TextBox'tan Veri Okuma IV

Okunan bu string değeri, label1 isimli etiketin metni (Text özelliği) olarak ayarlar.

Label'a Veri Yazma I

Label'a veri yazma, programın kullanıcıya geri bildirim vermesinin en temel yollarından biridir.



Label'a Veri Yazma II

Bir Label'ın metnini değiştirmek için de .Text özelliğine atama (=) operatörü ile yeni bir değer atanır.



Veri İşleme Aşamaları

Tipik bir etkileşim döngüsü: Veri Okuma -> Veri İşleme (Manipülasyon) -> Veri Yazma (Geri Bildirim) adımlarını içerir.

Uygulama Örneđi

Bu proje, veri okuma, metin birleřtirme (iřleme) ve Label'a veri yazma adımlarını bir araya getirir.



Proje Kurulumu I

Forma eklenecek kontroller: Bir Label ('Adınızı Girin:'), bir TextBox, bir Button ve ikinci bir Label.



Proje Kurulumu II

TextBox, txtIsim olarak isimlendirilir.
Button, btnSelamla olarak isimlendirilir.

Proje Kurulumu III

Sonucu gösterecek ikinci Label, lblSonuc olarak isimlendirilir.
Bu kontrolün Text özelliği başlangıçta boşaltılır.

Olay İşleyici Oluşturma

btnSelamla butonuna çift tıklanarak ilgili Click olayı metodu otomatik olarak oluşturulur.

Uygulama Kodu

Olay metodunun içine yazılacak kod: `lblSonuc.Text = 'Merhaba, ' + txtIsim.Text + '!'.`

Kodun Analizi I

Kodda kullanılan '+' operatörleri, statik metinler ile TextBox'tan okunan değişken metni birleştirir.



Kodun Analizi II

Girdi: txtIsim.Text özelliği okunur.
Çıktı: Birleştirilen sonuç, lblSonuc.Text özelliğine atanır.

Uygulamanın Test Edilmesi I

Program çalıştırılır, TextBox'a kullanıcı tarafından bir isim girilir.

Uygulamanın Test Edilmesi II

Butona basıldığında, lblSonuc etiketinde 'Merhaba, [Girilen isim]!' formatında sonucun yazdığı gözlemlenir.

Tam Etkileşim Döngüsü

Bu basit örnek, öğrencilerin kullanıcıdan aldıkları bir veriyi işleyip tekrar kullanıcıya sundukları ilk tam etkileşim döngüsüdür.



Önemli Notlar ve İpuçları

Kod yazarken dikkat edilmesi gereken temel C# sentaks kuralları mevcuttur.

C# ve Case-Sensitivity I

C# dili büyük/küçük harf duyarlıdır (case-sensitivity).
Bu, nesne adları, değişken adları ve özellik isimleri için hayati öneme sahiptir.

C# ve Case-Sensitivity II

label1.Text ile label1.text ifadeleri C# derleyicisi için tamamen farklı anlamlara gelir.
İkincisi (küçük harfle yazılan özellik) derleme hatası verecektir.

Noktalı Virgöl (;) Kullanımı I

Her komutun sonuna noktalı virgöl (;) konulmasının zorunluluğu vurgulanmalıdır.

Noktalı Virgül (;) Kullanımı II

Noktalı virgül, C#'ta tıpkı bir cümlenin sonuna konulan 'nokta' gibi işlev görür. Derleyiciye, yazılan komutun burada bittiğini bildirir.



Kontrol Özellikleri

Görsel programlamada en sık kullanılan özellik .Text özelliğidir. Bu özellik hem veri okumak hem de veri yazmak için kullanılır.

Metot İmza Özeti

Tüm olaylar, `private void KontrolAdi_OlayAdi(object sender, EventArgs e)` şablonuna uymaktadır.



Olay Gdml Programlamanın nemi

OGP paradigması, kullanıcı merkezli, etkileşimli ve modern yazılımlar geliřtirmenin temelini oluřturur.



Bu hafta 'olay' kavramını anladık, Button_Click olayını kullandık.
TextBox'tan veri okumayı ve Label'a veri yazmayı (.Text özelliği ile) öğrendik.

OGP, geleneksel prosedürel programlamadan farklı olarak, akış kontrolünü dışsal olaylara devreden reaktif bir mimari sunar.



Sonuç ve İleri Çalışmalar

Öğrenilen temel prensipler, daha karmaşık kontrollerin ve olayların yönetilmesi için bir altyapı sağlayacaktır. Kontrollerle Etkileşim ve Olay Yönetimi konuları derinleştirilecektir.



Kontrollerle Etkileşim ve Olay (Event) Yönetimi

Süleyman **EZDEMİR**

suleyman.ezdemir@giresun.edu.tr

